Red-Black Tree

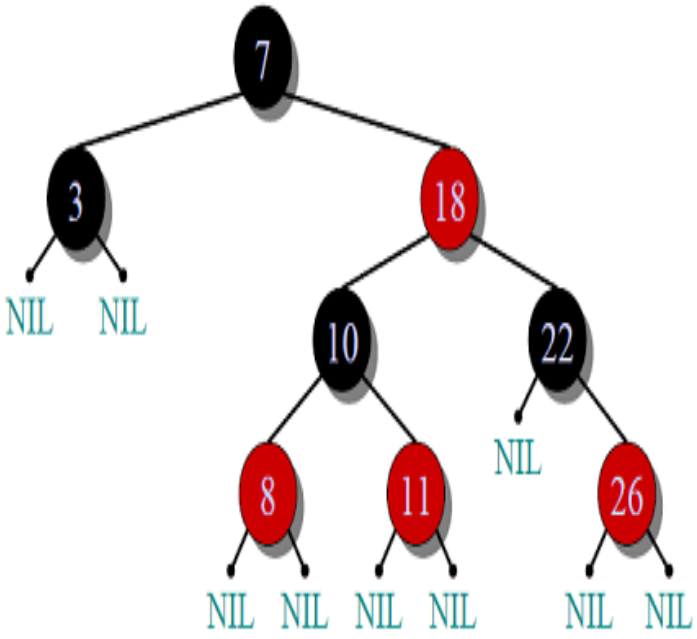
Red-Black Tree is a self-balancing Binary Search Tree (BST) where every node follows following rules.

1) Every node has a color either red or black.

2) Root of tree is always black.

3) There are no two adjacent red nodes (A red node cannot have a red parent or red child).

4) Every path from a node (including root) to any of its descendant NULL node has the same number of black nodes.



Why Red-Black Trees?

- Most of the BST operations (e.g., search, max, min, insert, delete.. etc) take O(h) time where h is the height of the BST. The cost of these operations may become O(n) for a skewed Binary tree. If we make sure that height of the tree remains O(Logn) after every insertion and deletion, then we can guarantee an upper bound of O(Logn) for all these operations. The height of a Red-Black tree is always O(Logn) where n is the number of nodes in the tree.

Comparison with AVL Tree

The AVL trees are more balanced compared to Red-Black Trees, but they may cause more rotations during insertion and deletion. So if your application involves many frequent insertions and deletions, then Red Black trees should be preferred. And if the insertions and deletions are less frequent and search is a more frequent operation, then AVL tree should be preferred over Red-Black Tree.

How does a Red-Black Tree ensure balance?

A chain of 3 nodes is nodes is not possible in Red-Black Trees.
Following are **NOT** Red-Black Trees

```
      30          30              30
     / \         / \             / \
   20  NIL     20   NIL        20   NIL
   / \         / \             / \
  10  NIL     10  NIL        10  NIL
Violates      Violates       Violates
Property 4.   Property 4     Property 3
```

Following are different possible Red-Black Trees with above 3 keys

```
       20                    20
      /  \                  /  \
   10    30              10    30
   / \   / \             / \   / \
NIL  NIL NIL NIL  NIL  NIL NIL NIL
```

In Red-Black tree, we use two tools to do balancing.

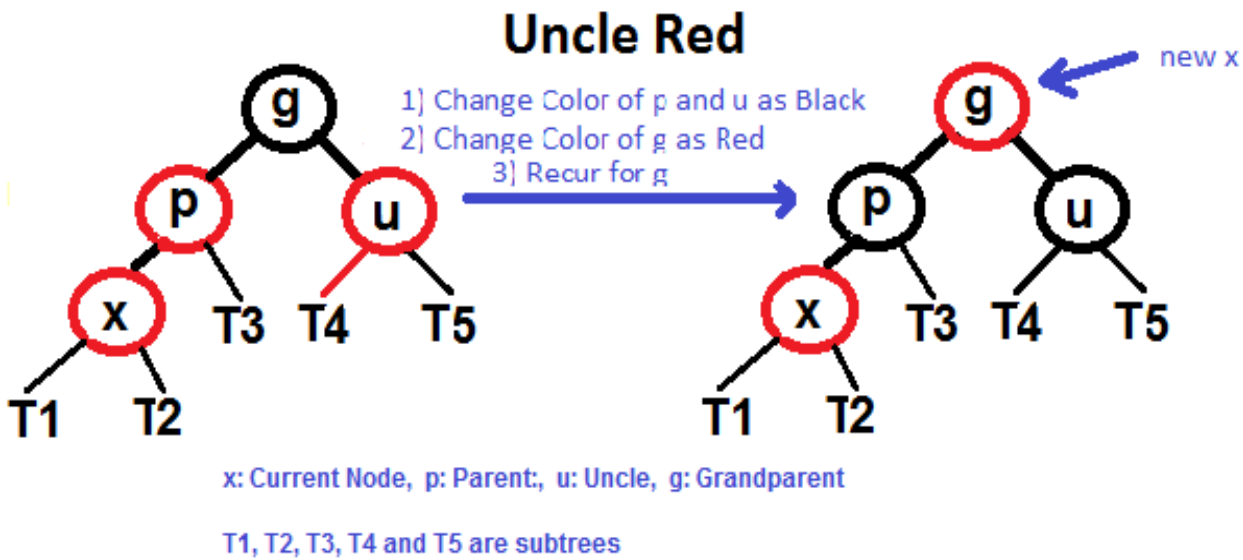
1) Recoloring
2) Rotation

- Color of a NULL node is considered as BLACK.
- Let x be the newly inserted node.

1) Perform <u>standard BST insertion</u> and make the color of newly inserted nodes as RED.

2) If x is root, change color of x as BLACK (Black height of complete tree increases by 1).

3) Do following if color of x's parent is not BLACK or x is not root.
....**a) If x's uncle is RED** (Grand parent must have been black from <u>property 4</u>)
.....**(i)** Change color of parent and uncle as BLACK.
.....**(ii)** color of grand parent as RED.
.....**(iii)** Change x = x's grandparent, repeat steps 2 and 3 for new x.



….**b) If x's uncle is BLACK**, then there can be four configurations for x, x's parent (**p**) and x's grandparent (**g**) (This is similar to AVL Tree)

……..**i)** Left Left Case (p is left child of g and x is left child of p)

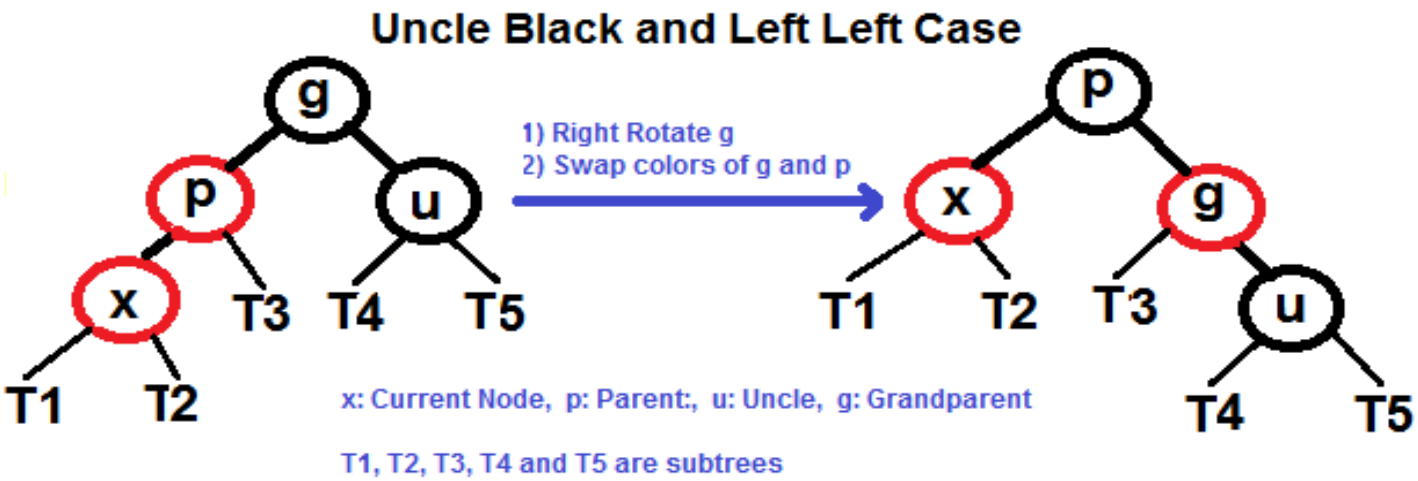
……..**ii)** Left Right Case (p is left child of g and x is right child of p)

……..**iii)** Right Right Case (Mirror of case i)

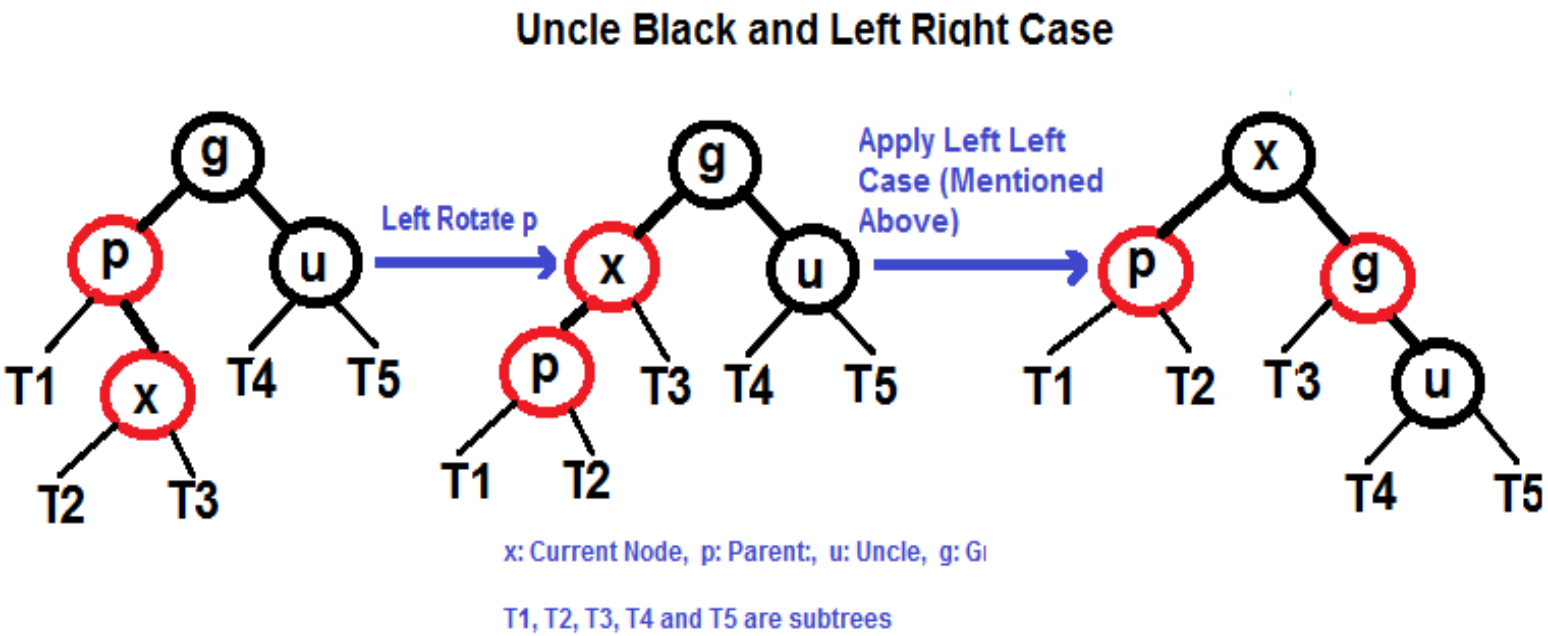
……..**iv)** Right Left Case (Mirror of case ii)

All four cases when Uncle is BLACK

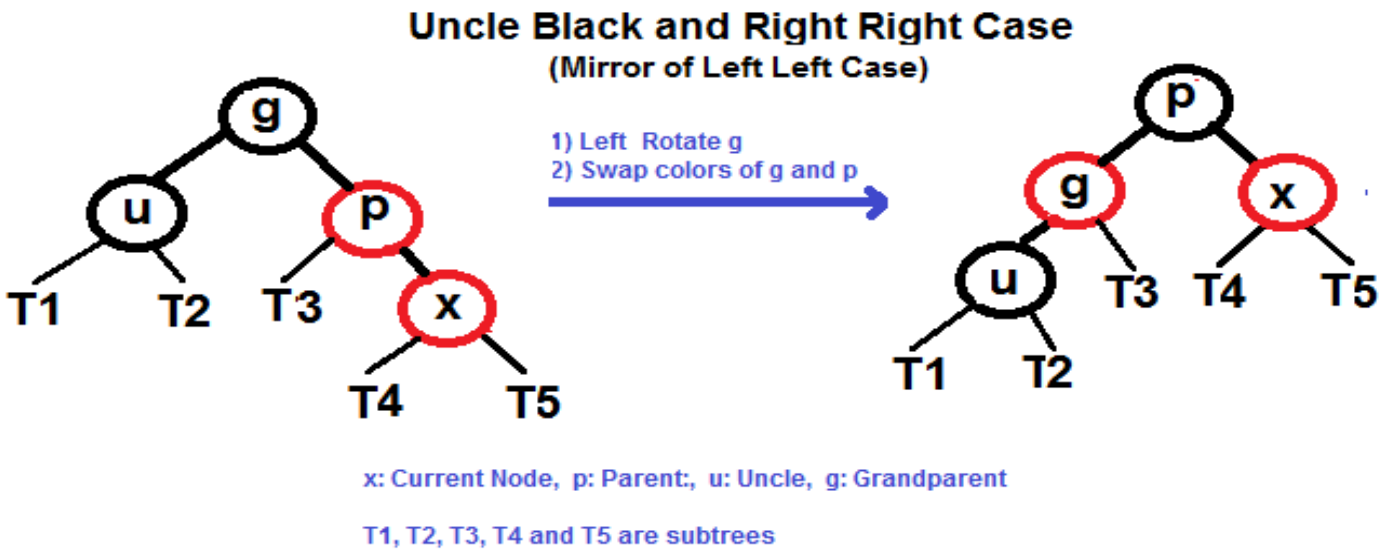
- **Left Left Case**



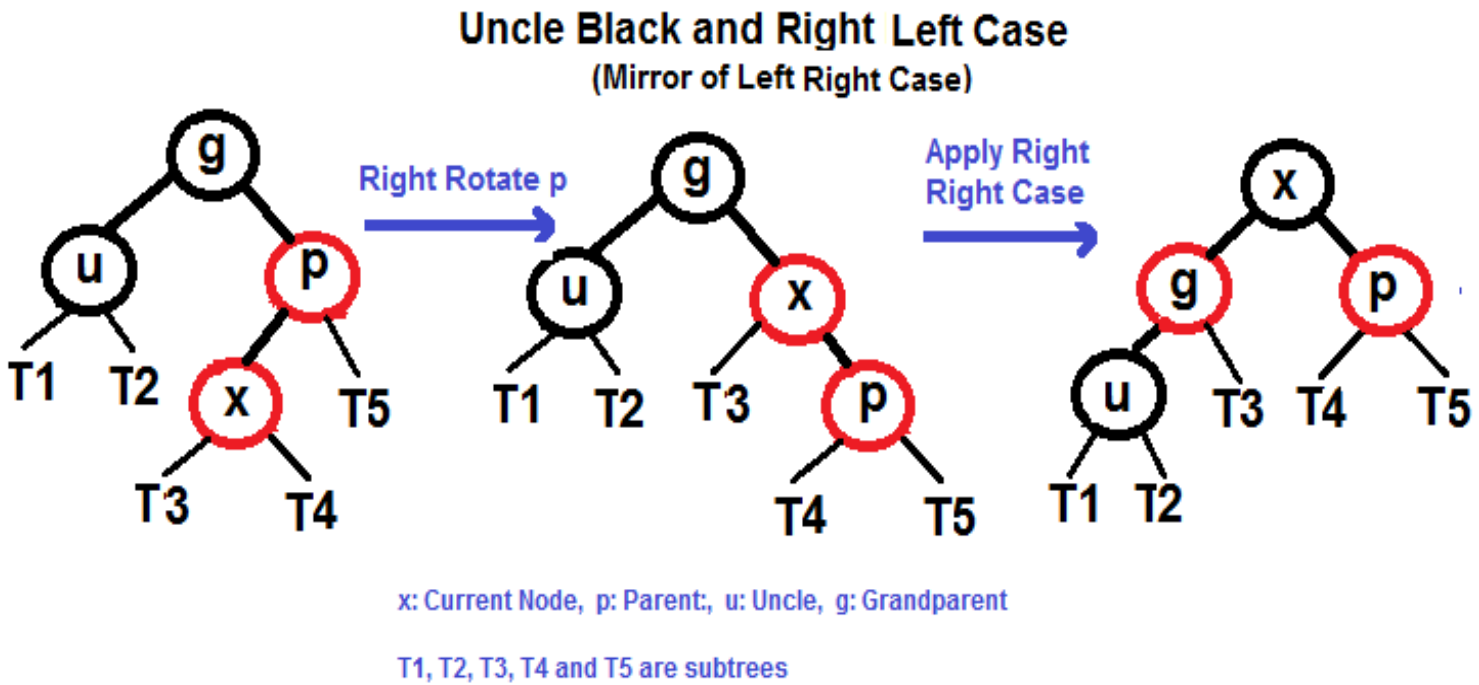
- **Left Right Case**



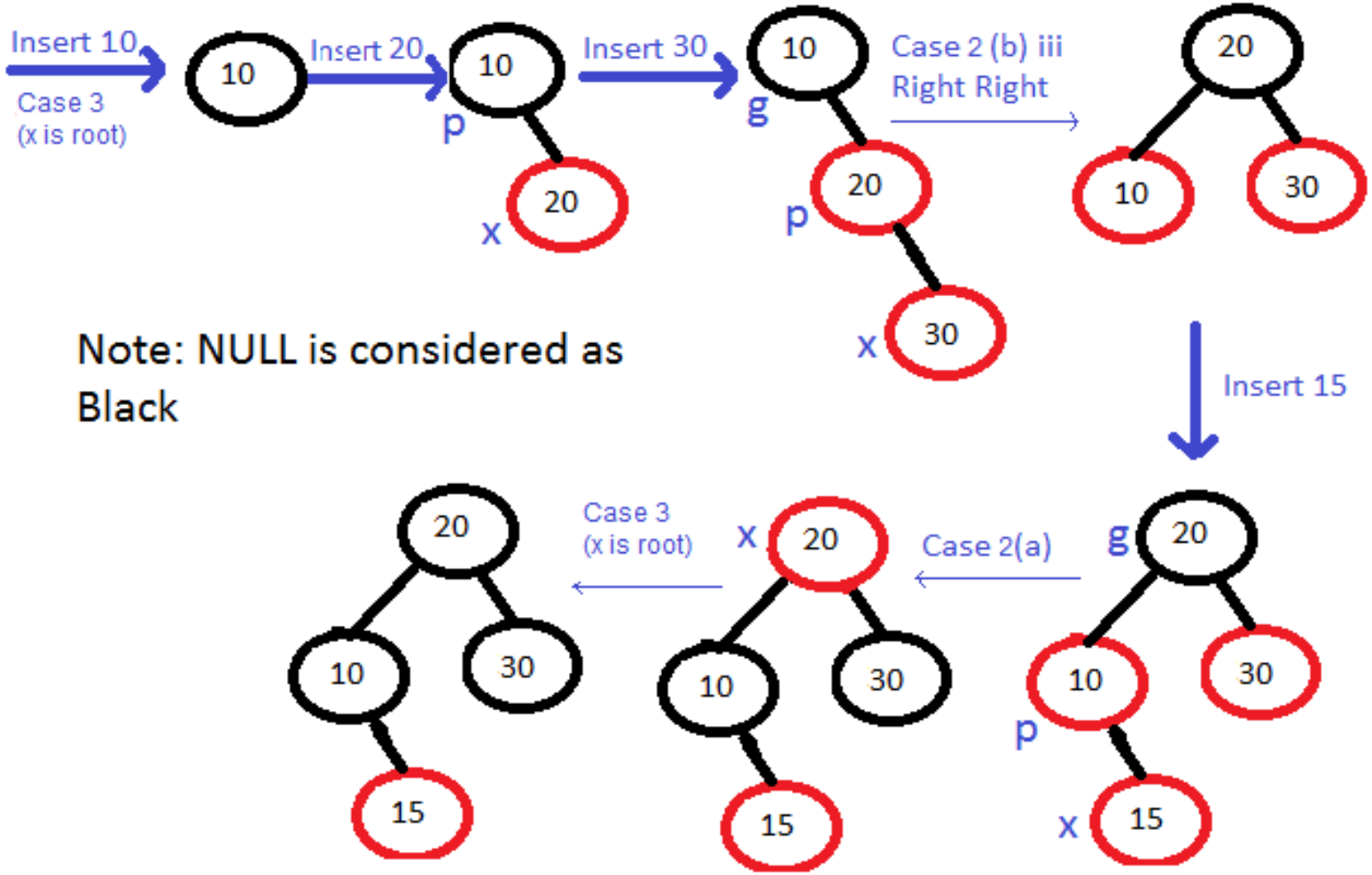
- **Right Right Case**



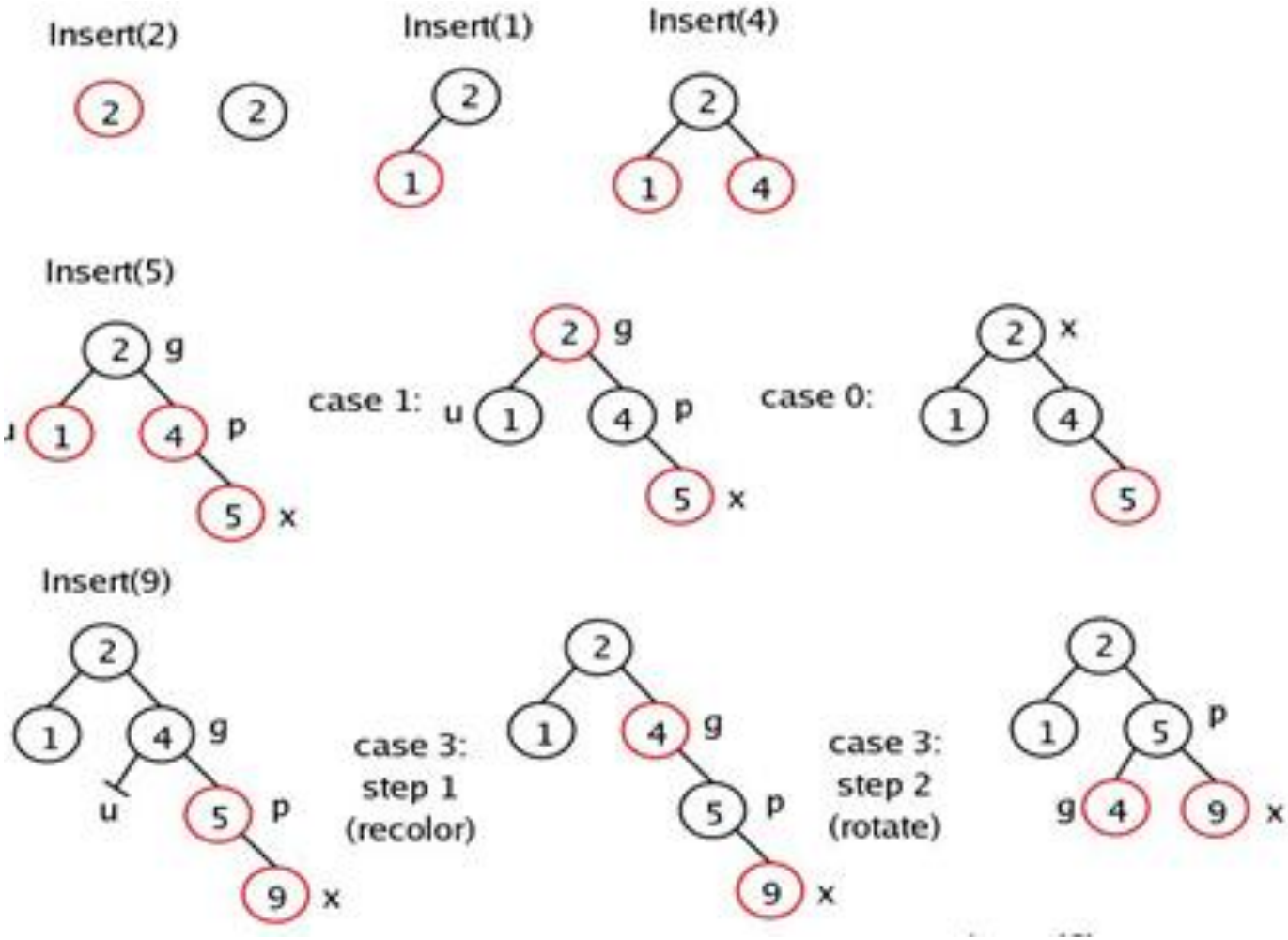
- **Right Left Case**



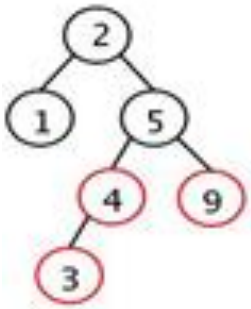
Insert 10, 20, 30 and 15 in an empty tree



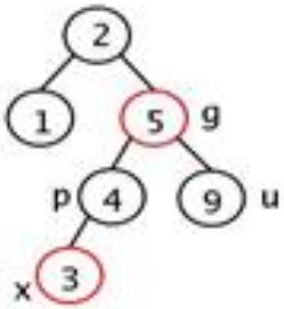
Note: NULL is considered as Black


Insert(2)
Insert(1)
Insert(4)
Insert(5)
case 1:
case 0:
Insert(9)
case 3:
step 1
(recolor)
case 3:
step 2
(rotate)

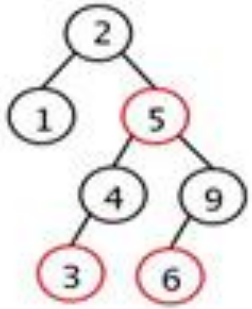
Insert(3)



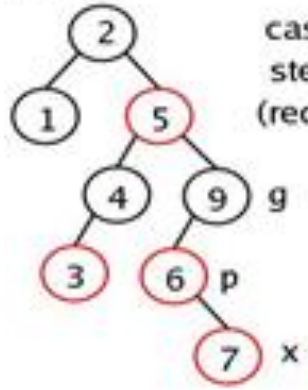
case 1:



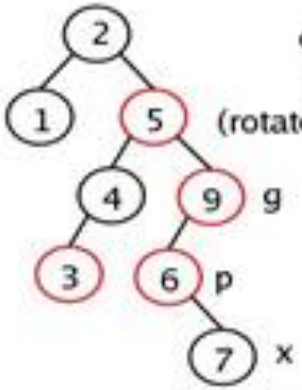
Insert(6)



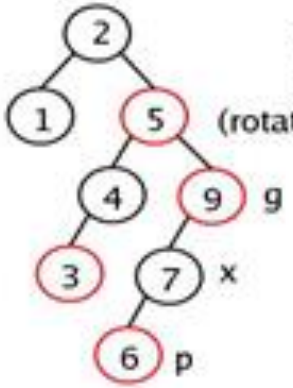
Insert(7)



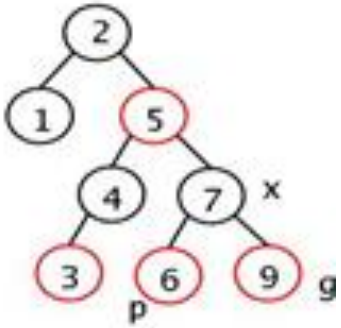
case 2:
step 1
(recolor)



case 2:
step 2
(rotate x about p)



case 2:
step 3
(rotate x about g)



Insert logic

- enum Color {RED, BLACK};
- bool color;
- int color;

```
struct Node
{
   int data;
   bool color;
   Node *left, *right, *parent;
};
```

```
void inorder (Node *root)
{
    if (root == NULL)
        return;

    inorder (root->left);
    cout << root->data << "  ";
    inorder (root->right);
}
```

```
Node* BSTInsert(Node* root, Node *pt)
{
   /* If the tree is empty, return a new node */
   if (root == NULL)
     return pt;

   /* Otherwise, recur down the tree */
   if (pt->data < root->data)
   {
      root->left  = BSTInsert(root->left, pt);
      root->left->parent = root;
   }
   else if (pt->data > root->data)
   {
      root->right = BSTInsert(root->right, pt);
      root->right->parent = root;
   }

   /* return the (unchanged) node pointer */
   return root;
}
```

```
RB_rules(Node *&root, Node *&pt)
{
   Node *parent_pt = NULL;
   Node *grand_parent_pt = NULL;

   while ((pt != root) && (pt->color != BLACK) &&   (pt->parent->color == RED))
   {
                        parent_pt = pt->parent;
                  grand_parent_pt = pt->parent->parent;

      /*  Case : A        Parent of pt is left child of Grand-parent of pt */
      if (parent_pt == grand_parent_pt->left)
      {

         Node *uncle_pt = grand_parent_pt->right;

         /* Case : 1         The uncle of pt is also red    Only Recoloring required */
         if (uncle_pt != NULL && uncle_pt->color == RED)
         {
            grand_parent_pt->color = RED;
            parent_pt->color = BLACK;
            uncle_pt->color = BLACK;
            pt = grand_parent_pt;
         }
           else
         {
```

```
/* Case : 2  pt is right child of its parent  Left-rotation required */
            if (pt == parent_pt->right)
            {
                rotateLeft(root, parent_pt);
                pt = parent_pt;
                parent_pt = pt->parent;
            }

 /* Case : 3 pt is left child of its parent   Right-rotation required */
            rotateRight(root, grand_parent_pt);
            swap(parent_pt->color, grand_parent_pt->color);
            pt = parent_pt;
        }
    }
```

```
/* Case : B
        Parent of pt is right child of Grand-parent of pt */
      else
      {
         Node *uncle_pt = grand_parent_pt->left;

  /*  Case : 1 The uncle of pt is also red Only Recoloring required */

                    if ((uncle_pt != NULL) && (uncle_pt->color == RED))
         {
            grand_parent_pt->color = RED;
            parent_pt->color = BLACK;
            uncle_pt->color = BLACK;
            pt = grand_parent_pt;

else
         {
}
```

/* Case : 2 pt is left child of its parent Right-rotation required */

```
if (pt == parent_pt->left)
{
    rotateRight(root, parent_pt);
    pt = parent_pt;
    parent_pt = pt->parent;
}
```

/* Case : 3 pt is right child of its parent, Left-rotation required */

```
    rotateLeft(root, grand_parent_pt);
swap(parent_pt->color, grand_parent_pt->color);
pt = parent_pt;
}
```

```
root->color = BLACK;

RBTree tree;

    tree.insert(7);
```

```
void rotateLeft(Node *&root, Node *&pt)
{
    Node *pt_right = pt->right;

    pt->right = pt_right->left;

    if (pt->right != NULL)
        pt->right->parent = pt;

    pt_right->parent = pt->parent;

    if (pt->parent == NULL)
        root = pt_right;

    else if (pt == pt->parent->left)
        pt->parent->left = pt_right;

    else
        pt->parent->right = pt_right;

    pt_right->left = pt;
    pt->parent = pt_right;
}
```

```
void rotateRight(Node *&root, Node *&pt)
{
    Node *pt_left = pt->left;

    pt->left = pt_left->right;

    if (pt->left != NULL)
        pt->left->parent = pt;

    pt_left->parent = pt->parent;

    if (pt->parent == NULL)
        root = pt_left;

    else if (pt == pt->parent->left)
        pt->parent->left = pt_left;

    else
        pt->parent->right = pt_left;

    pt_left->right = pt;
    pt->parent = pt_left;
}
```

Insertion Vs Delete

Insert	Delete
Recoloring and rotations are used	Recoloring and rotations are used
We check color of uncle to decide the appropriate case	***We check color of sibling*** to decide the appropriate case.
Main property that violates after insertion is two consecutive red.	Main violated property is, change of black height in subtrees as deletion of a black node may cause reduced black height in one root to leaf path.

When a black node is deleted and replaced by a black child, the child is marked as ***double black***. The main task now becomes to convert this double black to single black.

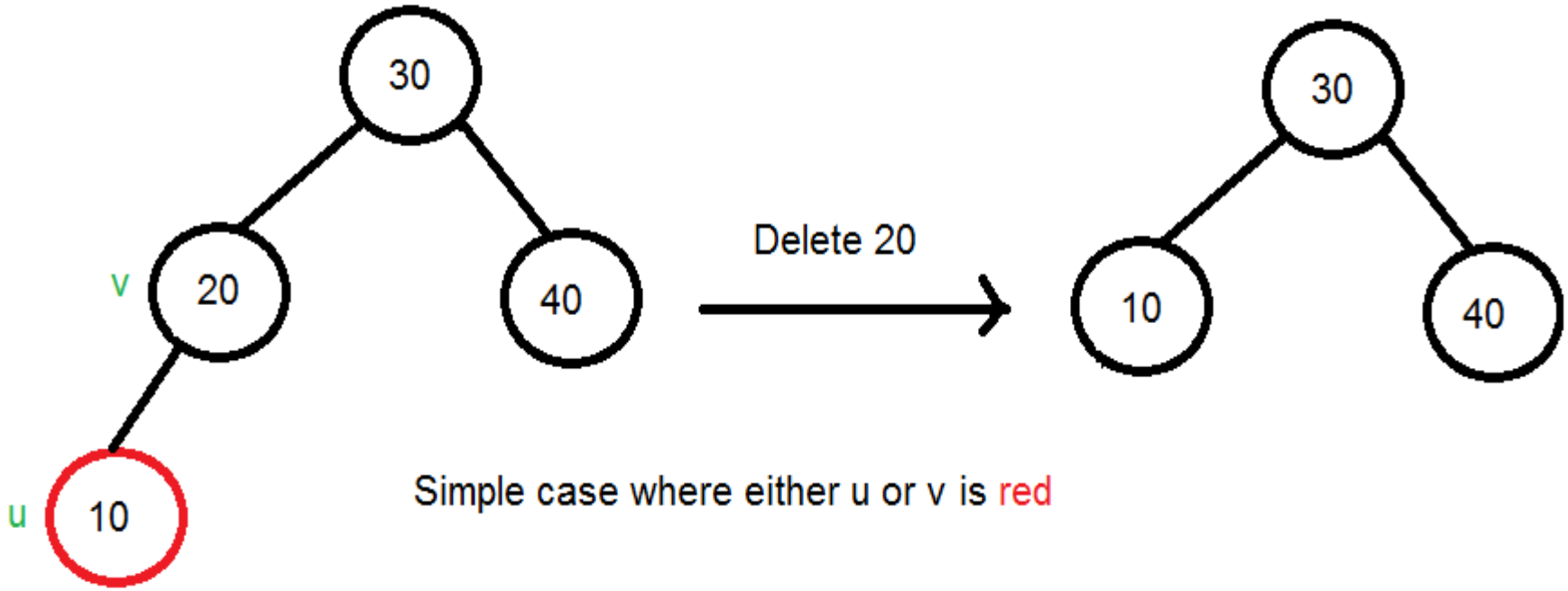
Deletion Steps

1) Perform standard BST delete.

(Delete node which is either leaf or has only one child. Successor will be used to replace.)

2) Case 1. If either u or v is red-

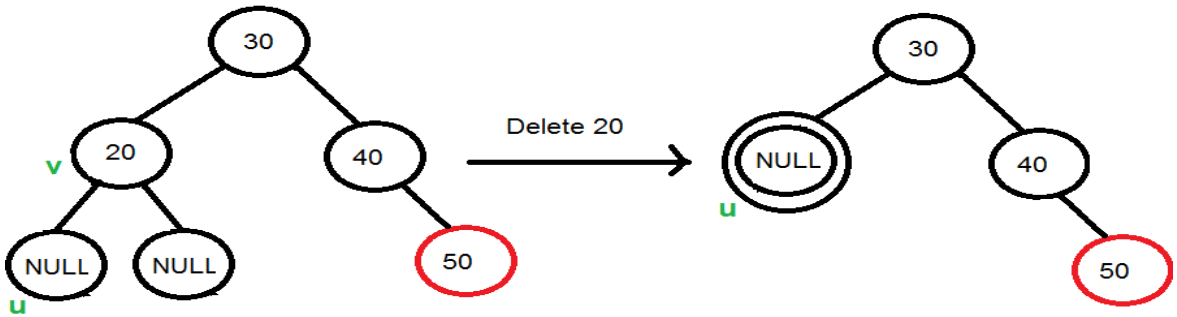
we mark the replaced child as black (No change in black height). Note that both u and v cannot be red as v is parent of u and two consecutive reds are not allowed in red-black tree.



Simple case where either u or v is red

- **If Both u and v are Black**

Color u as double black. Now our task reduces to convert this double black to single black. Note that If v is leaf, then u is NULL and color of NULL is considered as black. So the deletion of a black leaf also causes a double black.



When 20 is deleted, it is replaced by a NULL, so the NULL becomes double black.
Note that deletion is not done yet, this double black must become single black

while the current node u is double black and it is not root. Let sibling of node be **s**.

Case1:**If sibling s is black and at least one of sibling's children is red**

Case2: **If sibling is black and its both children are black**

Case3:**If sibling is red**

.**(a): If sibling s is black and at least one of sibling's children is red**, perform rotation(s). Let the red child of s be **r**. This case can be divided in four subcases depending upon positions of s and r.

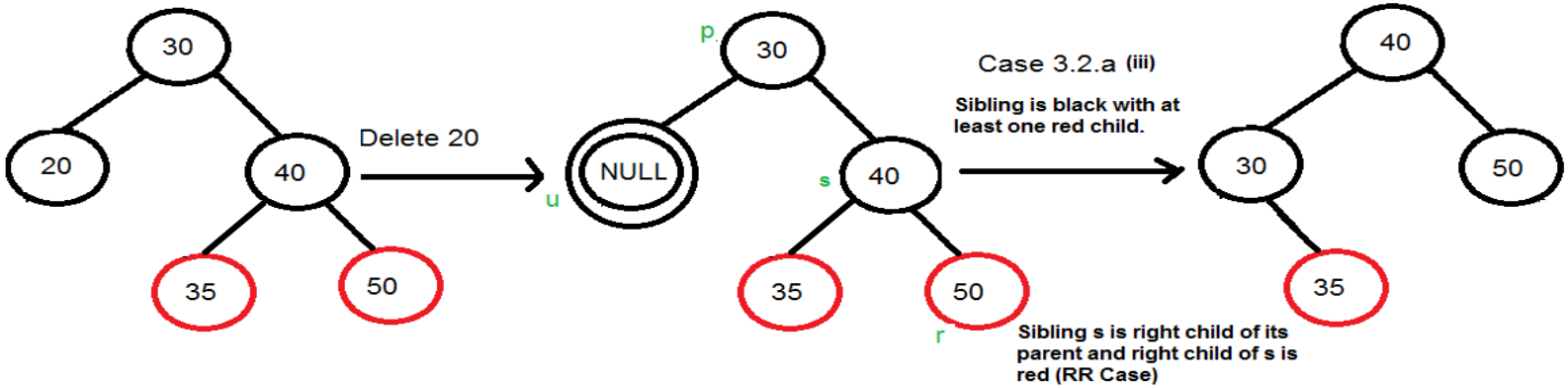
……..**(i)** Left Left Case (s is left child of its parent and r is left child of s or both children of s are red). This is mirror of right right case.

……..**(ii)** Left Right Case (s is left child of its parent and r is right child). This is mirror of right left.

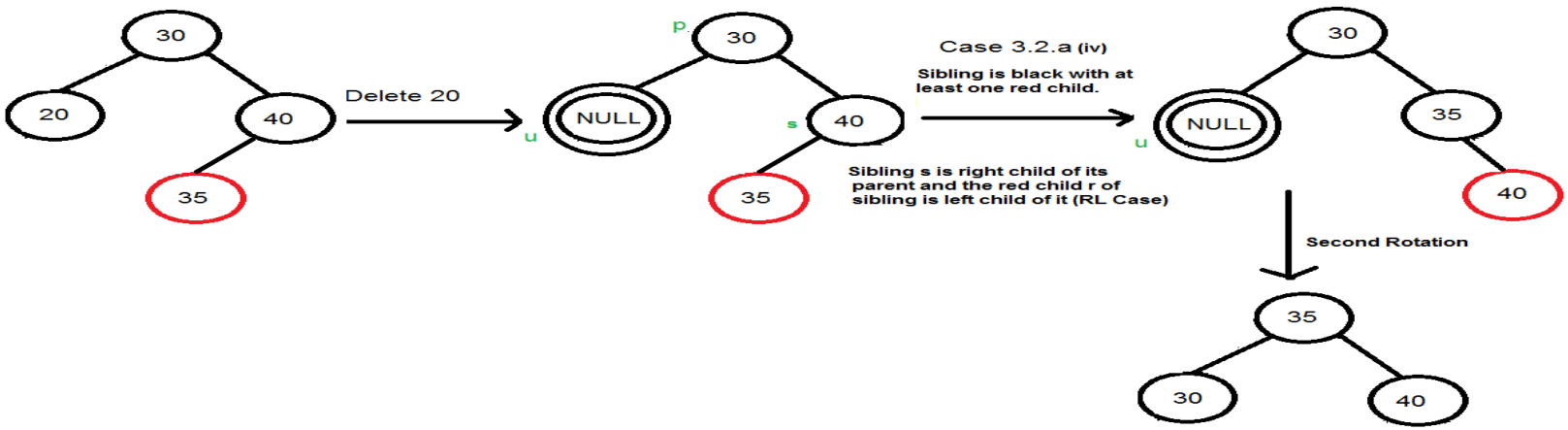
……..**(iii)** Right Right Case (s is right child of its parent and r is right child of s or both children of s are red)

……..**(iv)** Right Left Case (s is right child of its parent and r is left child of s)

(iii) Right Right Case (s is right child of its parent and r is right child of s or both children of s are red)

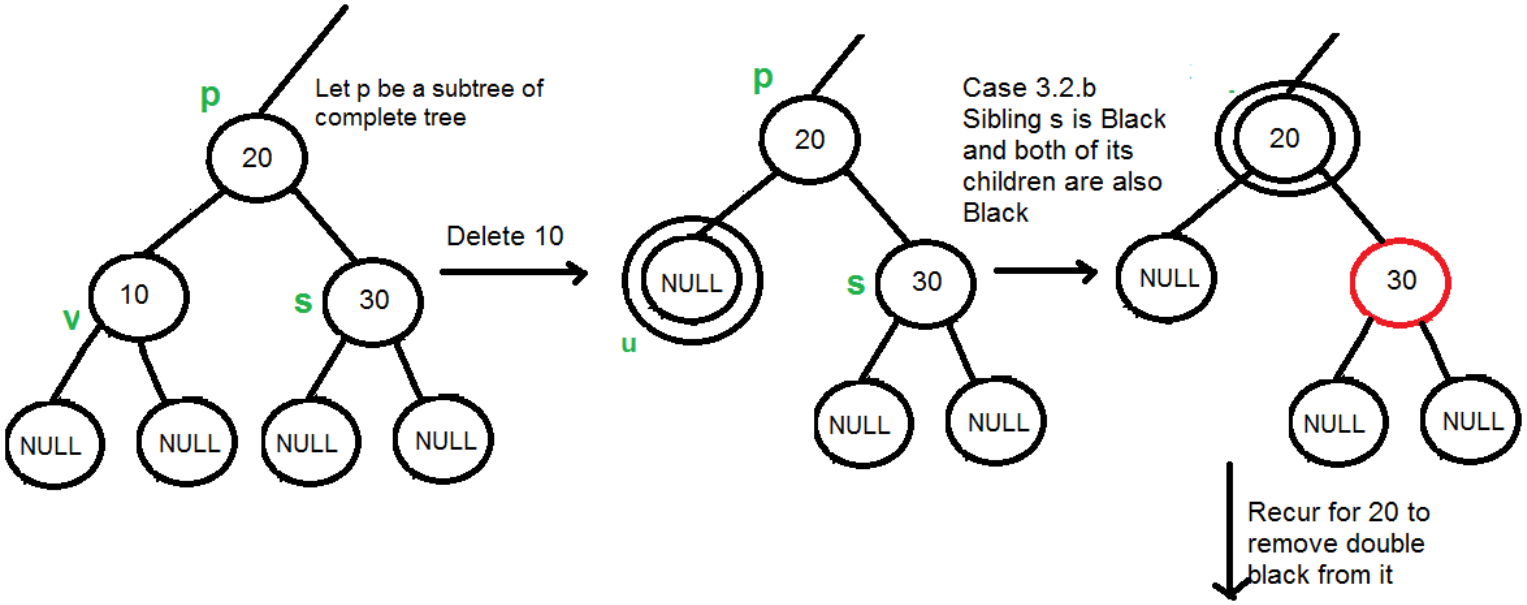


(iv) Right Left Case (s is right child of its parent and r is left child of s)



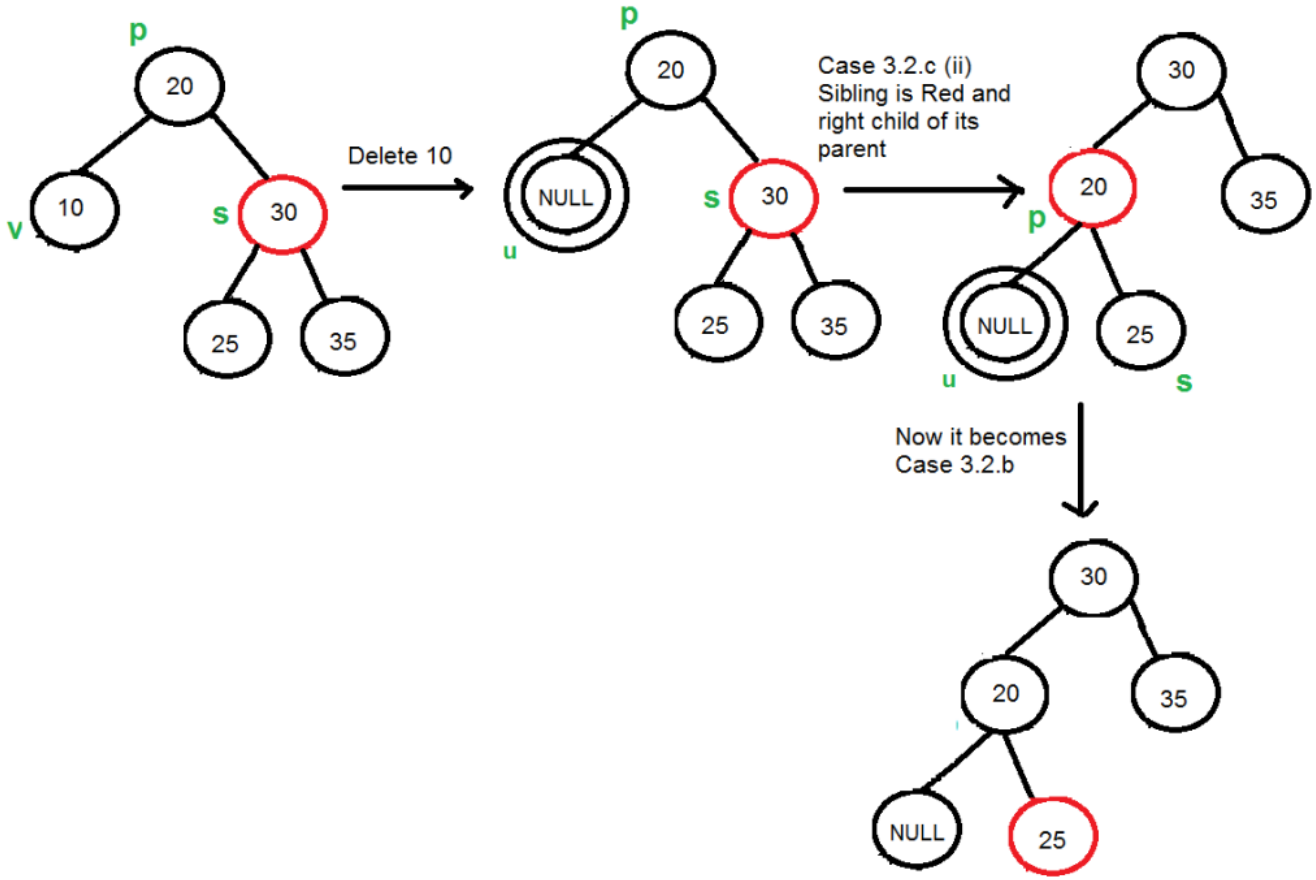
.**(b): If sibling is black and its both children are black**, perform recoloring, and recur for the parent if parent is black.

In this case, if parent was red, then we didn't need to recur for prent, we can simply make it black (red + double black = single black)



…..(c): If sibling is red,

perform a rotation to move old sibling up, recolor the old sibling and parent. The new sibling is always black



- If u is root, make it single black and return (Black height of complete tree reduces by 1).

```
Node * finduncle() {
   // If no parent or grandparent, then no uncle
   if (parent == NULL or parent->parent == NULL)
     return NULL;

   if (parent->isOnLeft())
     // uncle on right
     return parent->parent->right;
   else
     // uncle on left
     return parent->parent->left;
 }
```

```
Node * findsibling() {
   // sibling null if no parent
   if (parent == NULL)
     return NULL;

   if (isOnLeft())
     return parent->right;

   return parent->left;
 }
```

```
bool hasRedChild() {
   return (left != NULL and left->color == RED) or
          (right != NULL and right->color == RED);
 }
```

- bool isOnLeft() { return this == parent->left; }
- int isOnLeft() { return current == parent->left; }

```
void moveDown(Node *nParent) {
  if (parent != NULL) {
    if (isOnLeft()) {
      parent->left = nParent;
    } else {
      parent->right = nParent;
    }
  }
  nParent->parent = parent;
  parent = nParent;
}
```

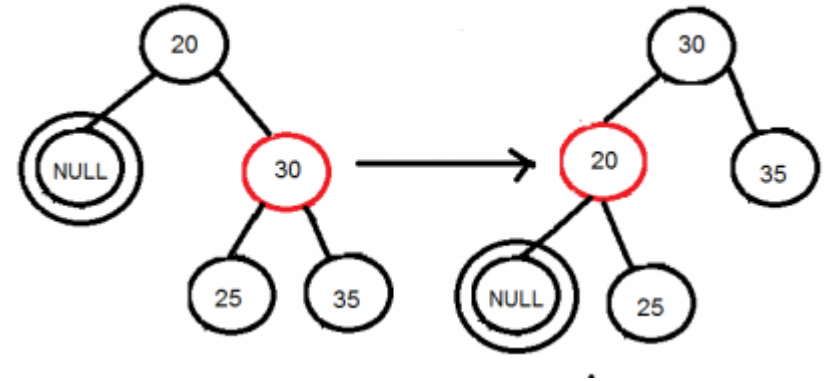
```
void leftRotate(Node *x) {
   // new parent will be node's right child
   Node *nParent = x->right;

   // update root if current node is root
   if (x == root)
     root = nParent;

   x->moveDown(nParent);

   // connect x with new parent's left element
   x->right = nParent->left;
   // connect new parent's left element with node
   // if it is not null
   if (nParent->left != NULL)
     nParent->left->parent = x;

   // connect new parent with x
   nParent->left = x;
 }
```



```
void rightRotate(Node *x) {
   // new parent will be node's left child
   Node *nParent = x->left;

   // update root if current node is root
   if (x == root)
     root = nParent;

   x->moveDown(nParent);

   // connect x with new parent's right element
   x->left = nParent->right;
   // connect new parent's right element with node
   // if it is not null
   if (nParent->right != NULL)
     nParent->right->parent = x;

   // connect new parent with x
   nParent->right = x;
 }
```

```
void swapColors(Node *x1, Node *x2) {
    COLOR temp;
    temp = x1->color;
    x1->color = x2->color;
    x2->color = temp;
  }
```

```
void swapValues(Node *u, Node *v) {
    int temp;
    temp = u->val;
    u->val = v->val;
    v->val = temp;
  }
```